

LAPORAN WORKSHOP DevOps

Bab 3 - Docker Network, Volume, Bind Mount, tmpfs, dan Compose



Nama : Fitria Sari Ayuningtyas

NRP : 3126640013

Dosen Pengajar : Dr. Ferry Astika Saputra, ST, M.Sc

**PROGRAM STUDI TEKNIK INFORMATIKA
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA (PENS)
TAHUN 2026**

1. Tujuan

- Memahami komunikasi antar-container menggunakan user-defined bridge.
- Membandingkan named volume, bind mount, dan tmpfs.
- Menerapkan Docker Compose untuk aplikasi multi-container.
- Melakukan verifikasi dan troubleshooting container.

2. Lingkungan Praktikum

Komponen	Spesifikasi
OS	Ubuntu pada WSL2
Docker Engine	29.7.2
Docker Compose	v5.5.0
Web Server	Nginx Alpine
Backend	Flask
Database	PostgreSQL 16

3.

3.1 User-defined Bridge Network

```
asus@LAPTOP-OC46B530:~/docker-lab/bab-3$ docker network inspect lab-net
[
  {
    "Name": "lab-net",
    "Id": "4aa5180410859ac670e737636a335db132f2708e12c177388876b4ba580e2d6d",
    "Created": "2026-09-11T14:28:19.818206707+07:00",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv6": true,
    "EnableIPV6": false,
    "IPAM": {
      "Driver": "default",
      "Options": {},
      "Config": [
        {
          "Subnet": "172.20.0.0/16",
          "Gateway": "172.20.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Options": {},
    "Labels": {},
    "Containers": {
      "270803065da5579ffe34b3cfb293e358148a82ed78269fa118bc640cc1c66e72": {
        "Name": "server-b",
        "EndpointID": "386924d658caff060201e449097935740e3b34949697b3a6b85f4b8ecf98d98c",
        "MacAddress": "92:bb:7a:49:f2:55",
        "IPv4Address": "172.20.0.3/16",
        "IPv6Address": ""
      },
      "375710cae01e47605af43db8e1c885bc8b4277d55dc80bda475b614696345ffb": {
        "Name": "server-a",
        "EndpointID": "886dc4800ad070a89b8dd93bd5c1bd8f3d1c0cbb4e2a86b610fd3941c43fb65",
        "MacAddress": "7e:3d:de:bb:e8:3c",
        "IPv4Address": "172.20.0.2/16",
        "IPv6Address": ""
      }
    },
    "Status": {
      "IPAM": {
        "Subnets": [
          {
            "172.20.0.0/16": {
              "IPsInUse": 5,
              "DynamicIPsAvailable": 65531
            }
          }
        ]
      }
    }
  }
]
```

```
asus@LAPTOP-OC46B530:~/docker-lab/bab-3$ docker exec server-a ping -c 3 server-b
PING server-b (172.20.0.3): 56 data bytes
64 bytes from 172.20.0.3: seq=0 ttl=64 time=0.997 ms
64 bytes from 172.20.0.3: seq=1 ttl=64 time=0.149 ms
64 bytes from 172.20.0.3: seq=2 ttl=64 time=0.127 ms

--- server-b ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 0.127/0.424/0.997 ms
```

Analisis

User-defined bridge network lab-net berhasil menghubungkan container server-a dan server-b. Pengujian menggunakan nama container server-b berhasil melakukan resolusi nama menjadi alamat IP 172.20.0.3 dengan 0% packet loss. Hal ini menunjukkan bahwa container pada user-defined bridge dapat saling berkomunikasi menggunakan nama.

3.2 Named Volume

```
asus@LAPTOP-OC46B530:~/docker-lab/bab-3$ docker run --rm -v data-vol:/data alpine:3.20 cat /data/log.txt
Fri Sep 11 08:31:33 UTC 2026
Fri Sep 11 08:31:38 UTC 2026
Fri Sep 11 08:31:43 UTC 2026
Fri Sep 11 08:31:48 UTC 2026
Fri Sep 11 08:31:53 UTC 2026
Fri Sep 11 08:31:58 UTC 2026
Fri Sep 11 08:32:03 UTC 2026
Fri Sep 11 08:32:08 UTC 2026
Fri Sep 11 08:32:13 UTC 2026
Fri Sep 11 08:32:18 UTC 2026
Fri Sep 11 08:32:23 UTC 2026
Fri Sep 11 08:32:28 UTC 2026
Fri Sep 11 08:32:33 UTC 2026
Fri Sep 11 08:32:38 UTC 2026
Fri Sep 11 08:32:43 UTC 2026
Fri Sep 11 08:32:48 UTC 2026
Fri Sep 11 08:32:53 UTC 2026
```

```
asus@LAPTOP-OC46B530:~/docker-lab/bab-3$ ls -lh data-vol-backup.tar.gz
-rw-r--r-- 1 root root 1.2K Sep 11 16:19 data-vol-backup.tar.gz
asus@LAPTOP-OC46B530:~/docker-lab/bab-3$
```

Analisis

Data pada named volume data-vol tetap tersedia setelah container writer dihapus. File log.txt masih dapat dibaca melalui container baru yang menggunakan volume tersebut. Proses backup juga berhasil menghasilkan data-vol-backup.tar.gz, sehingga named volume dapat digunakan untuk penyimpanan data yang persisten dan proses pencadangan.

3.3 Bind Mount

```
asus@LAPTOP-OC46B530:~$ docker exec brave_davinci cat /usr/share/nginx/html/index.html
Hello from host!
asus@LAPTOP-OC46B530:~$ docker exec brave_davinci cat /usr/share/nginx/html/index.html
Hello after host change!
```

Analisis

Bind mount memungkinkan direktori pada host dipetakan langsung ke dalam container. Setelah isi index.html pada host diubah, perubahan tersebut dapat langsung dibaca dari dalam container tanpa melakukan rebuild image. Hal ini menunjukkan bahwa bind mount cocok digunakan ketika container perlu mengakses file yang dikelola langsung dari host.

3.4 Tmpfs

```
asus@LAPTOP-OC46B530:~/docker-lab/bab-3/compose-lab$ docker exec tmpfs-restart cat /app/tmp/test.txt
data sementara
asus@LAPTOP-OC46B530:~/docker-lab/bab-3/compose-lab$ docker restart tmpfs-restart
tmpfs-restart
asus@LAPTOP-OC46B530:~/docker-lab/bab-3/compose-lab$ docker exec tmpfs-restart ls -la /app/tmp
total 4
drwxr-xr-x  2 root  root   40 Sep 11 11:47 .
drwxr-xr-x  3 root  root  4096 Sep 11 11:46 ..
asus@LAPTOP-OC46B530:~/docker-lab/bab-3/compose-lab$
```

Analisis

File test.txt berhasil dibuat dan dibaca sebelum container di-restart. Setelah restart, file tersebut tidak ditemukan pada direktori /app/tmp. Hasil ini menunjukkan bahwa tmpfs menyimpan data secara sementara dan tidak mempertahankan data setelah container restart.

3.5 Compose Multi-container

Docker Compose digunakan untuk menjalankan tiga service, yaitu Nginx sebagai web server, Flask sebagai backend, dan PostgreSQL sebagai database. Service web berada pada network frontend, sedangkan app terhubung ke frontend dan backend. PostgreSQL berada pada backend dan menggunakan named volume pg-data untuk penyimpanan data.

```
asus@LAPTOP-OC46B530:~/docker-lab/bab-3/compose-lab$ docker compose ps
NAME                IMAGE              COMMAND                SERVICE    CREATED        STATUS              PORTS
compose-lab-app-1   compose-lab-app    "python app.py"        app        10 minutes ago Up 10 minutes             
compose-lab-db-1    postgres:16-alpine "docker-entrypoint.s..." db         10 minutes ago Up 10 minutes (healthy)  5432/tcp
compose-lab-web-1   nginx:alpine       "/docker-entrypoint..." web        10 minutes ago Up 10 minutes              0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
```

```

asus@LAPTOP-OC46S330:~/docker-lab/bab-3/compose-lab$ curl -v http://localhost:8080/api/health
* Host localhost:8080 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
* Trying [::1]:8080...
* Established connection to localhost (::1 port 8080) from ::1 port 38332
* using HTTP/1.x
> GET /api/health HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/8.18.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Server: nginx/1.31.5
< Date: Fri, 11 Sep 2026 11:38:06 GMT
< Content-Type: application/json
< Content-Length: 39
< Connection: keep-alive
<
{"database":"connected","status":"ok"}
* Connection #0 to host localhost:8080 left intact

```

Hasil docker compose ps menunjukkan ketiga service berhasil berjalan dan PostgreSQL berstatus healthy. Pengujian /api/health melalui port 8080 menghasilkan HTTP 200 OK dengan status database connected. Hasil tersebut membuktikan bahwa Nginx berhasil meneruskan request ke Flask dan Flask berhasil terhubung ke PostgreSQL.

3. 6 Evaluasi dan Latihan Mandiri

1. Mengapa user-defined bridge lebih baik daripada default bridge?

User-defined bridge menyediakan name resolution antar-container secara otomatis dan memberikan konfigurasi network yang lebih terkontrol dibandingkan default bridge.

2. Apa risiko bind mount terhadap keamanan host?

Bind mount dapat memberikan container akses langsung ke file atau direktori host. Jika mount menggunakan mode writable atau direktori yang terlalu luas, container yang dikompromikan berpotensi mengubah atau merusak data host.

3. Apa perbedaan docker compose down dan docker compose down -v?

docker compose down menghentikan dan menghapus container serta network yang dibuat Compose, sedangkan docker compose down -v juga menghapus named volume yang digunakan oleh Compose. Karena itu, down -v dapat menyebabkan data persistent pada volume hilang.

4. Kapan depends_on dengan healthcheck lebih tepat?

depends_on dengan healthcheck lebih tepat ketika service bergantung pada service lain yang harus benar-benar siap digunakan, misalnya aplikasi Flask yang membutuhkan PostgreSQL sudah siap menerima koneksi.

5. Bagaimana strategi backup volume database produksi?

Backup dilakukan secara berkala menggunakan mekanisme yang konsisten dengan database, disimpan pada lokasi terpisah, dan diuji dengan proses restore secara berkala. Backup juga sebaiknya memiliki retensi dan perlindungan akses yang sesuai.

4. Analisa

- **Masalah dan Diagnosis**

Pada saat pemeriksaan log menggunakan docker compose logs --tail 50, ditemukan pesan dari Nginx bahwa file default.conf tidak dapat dimodifikasi karena menggunakan read-only file system. Setelah diperiksa pada compose.yaml, file konfigurasi Nginx memang di-mount menggunakan opsi :ro (read-only). Meskipun pesan tersebut muncul, service Nginx tetap berjalan dengan baik dan pengujian curl http://localhost:8080/api/health menghasilkan status 200 OK. Dengan demikian, masalah tersebut tidak menghambat fungsi aplikasi.

- **Risiko Keamanan dan Operasional**

Penggunaan bind mount dapat menjadi risiko apabila container diberikan akses tulis terhadap direktori host yang terlalu luas. Jika container mengalami kompromi, data pada host yang ter-mount berpotensi diubah atau dihapus. Selain itu, password database pada file Compose masih ditulis secara langsung sehingga kurang aman apabila konfigurasi dibagikan atau digunakan pada lingkungan production.

- **Rekomendasi untuk production-like environment**

Untuk lingkungan production-like, credential database sebaiknya tidak ditulis langsung pada file Compose, tetapi dikelola menggunakan environment variable yang aman atau secrets. Bind mount juga sebaiknya dibatasi hanya pada direktori yang diperlukan dan menggunakan mode read-only apabila container hanya membutuhkan akses baca. Aplikasi Flask sebaiknya menggunakan production WSGI server seperti Gunicorn, bukan development server. Versi image Docker juga sebaiknya dipin agar deployment lebih konsisten dan dapat direproduksi.

5. Kesimpulan

Praktikum Bab 3 berhasil menerapkan user-defined bridge network, named volume, bind mount, tmpfs, dan Docker Compose. Pengujian menunjukkan bahwa container pada user-defined network dapat melakukan komunikasi menggunakan nama container, named volume mempertahankan data setelah container dihapus, bind mount meneruskan perubahan file host, dan tmpfs tidak mempertahankan data setelah restart. Docker Compose juga berhasil menjalankan Nginx, Flask, dan PostgreSQL dengan healthcheck database yang berhasil.